NLP Assignment 2 Report

Muhammad Umer Siddiqui
ms08702@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Abdullah Shaikh
as09245@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Arqish Zaria
az09714@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Abstract

Keywords

Natural Language Processing, Artificial Neural Network, Sentiment Classification, Backpropagation, One-Hot Encoding, ReLU, Sigmoid, Binary Cross Entropy

1 Question 3: Artificial Neural Network for Sentiment Classification

1.1 Approach

1.1.1 Dataset Preparation. We loaded the synthetic dataset which contains 5000 training and 500 test samples. Each sample has a sentence made up of anonymized tokens like `word_1`, `word_2` and a binary label (0 or 1). We cleaned the text by converting to lowercase and removing punctuation, then tokenized each sentence by splitting on spaces. A vocabulary of 300 unique words was built from the training set only. Each sentence was then converted into a one-hot encoded vector of size 300.

1.1.2 Network Architecture. We implemented a simple feedforward ANN with one hidden layer from scratch using only NumPy:

- Input layer: 300 neurons (vocabulary size)
- Hidden layer: 128 neurons with ReLU activation
- Output layer: 1 neuron with Sigmoid activation

1.1.3 Forward Propagation. The forward pass computes:

$$Z_1 = X \cdot W_1 + b_1 \tag{1}$$

$$A_1 = \text{ReLU}(Z_1) \tag{2}$$

$$Z_2 = A_1 \cdot W_2 + b_2 \tag{3}$$

$$A_2 = \text{Sigmoid}(Z_2) \tag{4}$$

1.1.4 Loss Function. We used Binary Cross Entropy loss:

$$L = -\frac{1}{n} \sum [y \cdot \log(A_2) + (1 - y) \cdot \log(1 - A_2)] \tag{5}$$

1.1.5 Backpropagation. We computed gradients layer by layer using the chain rule.

Output Layer Gradients. Starting from the loss, we differentiate with respect to A_2:

$$\frac{\partial L}{\partial A_2} = -\frac{y}{A_2} + \frac{1 - y}{1 - A_2} \tag{6}$$

The derivative of Sigmoid is:

$$\frac{\partial A_2}{\partial Z_2} = A_2(1 - A_2) \tag{7}$$

Applying the chain rule:

$$\delta Z_2 = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} = \left(-\frac{y}{A_2} + \frac{1 - y}{1 - A_2} \right) \cdot A_2(1 - A_2) = A_2 - y \tag{8}$$

Since $Z_2 = A_1 \cdot W_2 + b_2$:

$$\delta W_2 = \frac{1}{n} A_1^T \cdot \delta Z_2 \tag{9}$$

$$\delta b_2 = \frac{1}{n} \sum \delta Z_2 \tag{10}$$

Hidden Layer Gradients. Propagating the error back through W_2:

$$\delta A_1 = \delta Z_2 \cdot W_2^T \tag{11}$$

The derivative of ReLU is:

$$\text{ReLU}'(Z_1) = \begin{cases} 1 & \text{if } Z_1 > 0 \\ 0 & \text{if } Z_1 \leq 0 \end{cases} \tag{12}$$

Applying the chain rule:

$$\delta Z_1 = \delta A_1 \times \text{ReLU}'(Z_1) \tag{13}$$

Since $Z_1 = X \cdot W_1 + b_1$:

$$\delta W_1 = \frac{1}{n} X^T \cdot \delta Z_1 \tag{14}$$

$$\delta b_1 = \frac{1}{n} \sum \delta Z_1 \tag{15}$$

Weight Updates. Weights were updated using gradient descent:

$$W = W - \alpha \cdot \delta W \tag{16}$$

$$b = b - \alpha \cdot \delta b \tag{17}$$

where $\alpha = 0.1$ is the learning rate.

1.2 Results

Table 1: Evaluation Results on Test Set

Metric	Value
Accuracy	52.60%
Precision (Positive)	0.5149
Recall (Positive)	0.9641
F1 Score (Positive)	0.6713
Precision (Negative)	0.7000
Recall (Negative)	0.0843
F1 Score (Negative)	0.1505
Macro F1	0.4109

Table 2: Confusion Matrix

	Predicted 0	Predicted 1
Actual 0	21	228
Actual 1	9	242

1.2.1 Confusion Matrix. The model predicts almost everything as class 1, as shown by the high recall for the positive class (0.9641) and near-zero recall for the negative class (0.0843). This imbalance is reflected in the large off-diagonal entry of 228 false positives against only 21 true negatives.

1.3 Challenges

The main challenge was the nature of the dataset. All 300 words of the vocabulary appear in both positive and negative sentences. This means there is no meaningful signal for the model to learn from, the words do not distinguish between the two classes. Secondaly, one-hot encoding produces sparse vectors where around 97% of values are zeros, and the model ends up predicting randomly. The loss stayed at 0.6931 throughout training.

1.4 Potential Improvements

- Using TF-IDF instead of one-hot encoding to give words different weights based on their discriminative power
- Adding more hidden layers to increase model capacity
- Using a real labeled sentiment dataset instead of synthetic anonymized tokens, where word choice actually correlates with the label.

1.5 Conclusion of Question 3

This part implemented a feedforward ANN from scratch using NumPy for binary sentiment classification. The network was trained correctly and the backpropagation derivations were verified, but the model could not learn because the synthetic dataset provides no discriminative signal: all vocabulary words appear with similar frequencies in both classes. The result is a model that collapses to predicting the majority class, achieving 52.60% accuracy with a loss fixed at 0.6931 throughout training.

2 Overall Conclusion